



United International University

**Group No : 03 (Xenon) (Title: Mock Competitive
Programming API)**

Course Code: CSE 4495 Section : B

Course Name: Software Testing and Quality Testing Assurance

Submitted By

Md. Khademul Islam Nahin	011221282
Mahi Hoque	011221283
Irfanuzzaman Montasir	011221276
Md. Abdullah Al Imran	011221326
Abu Sufian Robin	0112230721

Submitted To

Mobaswirul Islam

Lecturer Of CSE Dept.

United International University

Submission Date: 03 September, 2025

Structure of the Testing Document

1. Test Plan.....	3
2 . Test Cases.....	4
3. Defects.....	19
Defect ID: D-500-001.....	19
Defect ID: D-404-001.....	20
Defect ID: D-400-001.....	21
Defect ID: D-200-001.....	22
Defect ID: D-403-001.....	24
Defect ID: D-400-002.....	25
4. Risk and Recommendations.....	26
1. Weak Registration Process (Evidence).....	26
2. Insecure Data Exposure.....	26
3. Duplicate Email Registration Allowed.....	26
4. SQL injection.....	27

1. Test Plan

Purpose: Verify that all endpoints in the Mock Competitive Programming API work as intended.

Scope: Covers authentication, user management, contests, problems, and submissions.

Test Environment: Node.js Express server (app.js), Postman (VS Code extension).

Tools: Postman for API testing, JWT tokens for auth, local in-memory DB for mock data.

Out of Scope: External integrations, performance testing.

2 . Test Cases

Title:	Register, Login, Get Profile					
Test Case ID	TC-01					
Tester Name	<i>Abu Sufian Robin</i>					
Test case Description	To Check User Authentication functionality of Mock Competitive Programming API					
Pre-Condition	Server running; no existing user with same username; Bearer Token Provided when logged in					
Test Scenario	Checking that after entering the correct username, password and other credentials , the user can login using generated Bearer Token.					
Test Case ID	Test Steps	Expected Result	Actual Result	Status	Comments	Evidence
TC-01	1. POST /auth/register with {username, password, email}	Step 1 → 201 Created + new user	<i>201 Created</i>	Pass	No issues found.	Register_User.PNG
	2. POST /auth/login with the same credentials	Step 2 → 200 OK + JWT token	<i>200 OK</i>	Pass		Auth_Login.PNG
	3. GET /auth/profile with returned JWT	Step 3 → 200 OK + correct user profile	<i>200 OK</i>	Pass		User Login.PNG
	4. GET /auth/profile with returned JWT	401 Invalid Token	<i>401 Unauthorized</i>	Pass	JWT verification error: JsonWebTokenError invalid signature	User Login_Error.PNG
	5. POST /auth/register without {username, password,}	400 — {"message": "Username and password are required"}	<i>400 Bad Request</i>	Pass	Issue Found	Register_Error.PNG

Title:	Basic User and System Operations					
Test Case ID	TC-02					
Tester Name	<i>Abu Sufian Robin</i>					
Test case Description	System admin can see all the users and find the specific user by id and sort the valid users by username & id for making search operation smoothly. The system gets the specific user rating by user id. The check system will perform the update user profile by the user's id after authentication.					
Pre-Condition	Server running, users exist in DB , Bearer Token Provided when system information update required.					
Test Scenario	Checking that after entering the endpoints, user id, username and other parameters , the user can update profile, see the user list and search operations using generated Bearer Token.					
Test Case ID	Test Steps	Expected Result	Actual Result	Status	Comments	Evidence
TC-02	1. Send GET /users	Returns 200 OK with a JSON array of users. Each object contains { id, username, email }. Default order by ID.	200 OK	Pass	No issues found.	Get User List.PNG
	2. Send GET /users?sortBy=username&order=asc	Returns 200 OK with JSON array sorted alphabetically by username	200 OK	Pass		Sort User.PNG
	3. Send GET /users/1	Returns 200 OK with JSON { id:3, username "sufian" }.	200 OK	Pass		Get a User by id.PNG
	4. Send PUT /users/2 with Authorization: Bearer <token> and body { " country " : " USA ",	Returns 200 OK with updated message " Profile updated successfully "	200 OK	Pass		Profile Updated by user id.PNG , Profile Update Test Script.PNG

	"organization": "MIT" }.					
	4. Send GET /users/3/rating	Returns 200 OK with JSON {id:3, rating: 1500}.	200 OK	Pass		user Rating Test.PNG , Users's Rating.PNG
	5. Send PUT /users/2 with Authorization: Bearer <token> and body { "country " : "USA", "organization": "MIT" }.	Returns 403 with error message "You can only update your own profile"	403 Forbidden	Pass	AssertionEr ror: expected response to have status code 200 but got 403	Profile Updated Error.PNG

Title:	Contest Creation & Management					
Test Case ID	TC-03					
Tester Name	Md. Khademul Islam Nahin					
Test case Description	Verify that an admin can successfully create a contest by providing valid details.					
Pre-Condition	Admin user is logged in and authenticated with a valid JWT token. API endpoint /contests are available and working.					
Test Scenario	In this phase, we tested contest creation and management features. Only admins can create contests with valid data, while invalid inputs or non-admin attempts are blocked. We verified adding problems, fetching all or upcoming contests, and ensured access control so only setters can view contest details.					
Test Case ID	Test Steps	Expected Result	Actual Result	Status	Comments	Evidence
	1. POST/Contests	201 Created	201 created	Passed		https://drive.google.com/file/d/1tE0o9E

TC-03	With {name,dateTime,duration}	+ new contest created				NnIA_ZnuRkr t4_sfBtrDkW qtuW/view?u sp=drive_link
	1.1 POST/Contests With {No token }	-No contest created	401 unauthoriz ed	Passed		https://drive.google.com/file/d/1W9Gx5IRpD254MBfSI6VtfvERwUbBYsOk/view?usp=drive_link
	1.2 POST/Contests With { invalid token provided	-No contest created	401 unauthoriz ed	passed		https://drive.google.com/file/d/1W9Gx5IRpD254MBfSI6VtfvERwUbBYsOk/view?usp=drive_link
	2.POST/contests/problems With {contestId,problemName ,description,output,cate gory,difficulty,timeline} + User authorization who is the creator of that contest.	201 Created + new problem added to contest	201 Created	Passed		https://drive.google.com/file/d/19-BLxYfe4hg-aOtekhGT3zfFnAZ8vcj8/view?usp=drive_link
	2.1 POST/contests/problems {No token or invalid token provided of an individual who wants to create a problem of his/her own contest}	No problem created of that contest	401 Unauthoriz ed	Passed		https://drive.google.com/file/d/1J4_41z_q9MwnQT9oQBjRQxOahpDgPeX8/view?usp=drive_link

2.2 POST/contests/problem s {another user who is not creator of that event but trying to create a problem of that event}	The problem of that event won't be created.	403 Forbidden	Passed		https://drive.google.com/file/d/1JhP4pCbEjGGARP301RFYoP1h8mQPXqbz/view?usp=drive_link
2.3 POST/contests/problem s Creator of that contest trying to post a problem of an invalid contest id, which does not exist.	A new problem of that event won't be created.	404 Not Found	Passed		https://drive.google.com/file/d/1J4JnUQyWzkkIK6a1XgJo9udVG9CvVcRo/view?usp=drive_link
2.4 POST/contests/problem s Trying to post a json object which one or many keys value are empty.	A new problem of that event won't be created.	400 Bad Request	Passed		https://drive.google.com/file/d/1aDDjqYkJS6nEfhATiwGK4hKrlouhdi01/view?usp=drive_link
3.GET /contests Here a user is trying to watch all the contests including past,present and upcoming,which are created.	All Contest will be shown.	200 Ok	Passed		https://drive.google.com/file/d/1NOGmN5UanjXGaFzX3TfDzn5HlnPvtHDI/view?usp=drive_link
4.GET /contests/upcoming Here a user is trying to watch all the contests which are upcoming.	All upcoming Contests will be shown.	200 Ok	Passed		https://drive.google.com/file/d/1rl6r23VGHAbNNyXZnlqrG_uSiHwTIZy4/view?usp=drive_link
5.GET /contests/:id	upcoming Contests will be shown in	200 Ok	passed		https://drive.google.com/file/d/1RdSzu17uGH1HYhfT

	If upcoming and user token username equals the setter, full details are returned.	details including the setter.				JoKAZLN1gzicBtdT/view?usp=drive_link
	5.1GET /contests/:id If past, details are returned without a setter.	Here a past contest will be shown which the correct user has created but his/her setter details won't be shown in the details section.	200 Ok	passed		https://drive.google.com/file/d/1puGREeXZ3yvQUwEOpa2RZtNpRQwzE8/view?usp=drive_link
	5.2GET /contests/:id A user who isn't a setter trying to see upcoming contest info.	Upcoming events won't be shown for him because he isn't the setter.	400 Bad Request	Passed		https://drive.google.com/file/d/1dachvij8dbxoYaB5Sh4zb7uVnIFgWux7/view?usp=drive_link
	5.3GET /contests/:id User trying to see a contest which is not exist.	The contest won't be shown because it does not exist.	404 Not Found	Passed		https://drive.google.com/file/d/1iMjFJZ122Jw8RlpMhZNPucaM3BZrarED/view?usp=drive_link

Title:	Contest participation
Test Case ID	TC-04
Tester Name	Mahi hoque
Test case Description	Joining contest, view participant list of valid contest and problem fetching
Pre-Condition	User account exists (registered + logged in).

Test Scenario	Users can join a contest only before it starts, view the list of participants, and access problems after joining. Attempts to join late or fetch problems without joining are denied.					
Test Case ID	Test Steps	Expected Result	Actual Result	Status	Comments	Evidence
TC-04	1.1. POST /contests/:id/join User who isn't contest creator, hasn't joined, and joining before start	"message": "User a joined contest 1", "contestId": 1(200)	200	Pass		https://drive.google.com/file/d/1NtbNnaAKX1VxcDOsfxRIWCWXFzDdNVLc/view?usp=drive_link
	1.2. POST /contests/:id/join Bearer token not provided	"message": "No token provided"(401)	401	Pass		https://drive.google.com/file/d/1FJ-bnvXS8w3J6NT3Q3pa8do499KLqAIS/view?usp=drive_link
	1.3. POST /contests/:id/join can't join own contest	"message": "You cannot join your own contest"(400)	400	Pass		https://drive.google.com/file/d/1tMPZzJ9TI_hUx98QtXMZPbAA7JCubCTi/view?usp=drive_link
	1.4. POST /contests/:id/join after content ended	"message": "Contest has already ended"(400)	400	Pass		https://drive.google.com/file/d/1nGBX80J_NU5ZBuDGcbFd2b8RanJ4rXzS/view?usp=drive_link
	1.5. POST /contests/:id/join joining multiple time	"message": "You have already joined this contest"(400)	400	Pass		https://drive.google.com/file/d/1E66t9pE0xmTYUF7HOy_17gsxPAD5NHZ3/view?usp=drive_link
	1.6 GET /contests/:id/participants valid contest id and contest has ended	{ "contestId": 4, "participants": [{ "id": 2, "username": "b" }] }(200)	200	Pass		https://drive.google.com/file/d/184mpwEisXE2yqK13BcDQpR_M0-KPZHMq/view?usp=drive_link

Title:	Problem Solving and Submissions					
Test Case ID	TC-05					
Tester Name	Md. Abdullah Al Imran					
Test case Description	To verify the functionality of problem listing, viewing problem details, submitting solutions, and fetching submissions for problems.					
Pre-Condition	- At least one contest must exist in the system. - At least one problem should be added to a contest. - At least one user account (setter) is logged in and authorized to create contests/problems. - At least one participant account exists for testing submissions. - Contest must be in past or ongoing state for problems to be visible.					
Test Scenario	The user can see the problem, contest list, submission list using his id.					
Test Case ID	Test Steps	Expected Result	Actual Result	Status	Comments	Evidence
TC-05	1. Send GET /problems request List all available problems	Returns list of problems from past contests (200 OK)	200 OK	Pass		List_Problems
	2. GET /problems/:id Send with valid problem ID to view problem details (id:1)	Returns problem details (200 OK)	200 OK	Pass		View_problem_with_valid_id
	2.1. View problem details with non existing ID (id:5)	Returns error 404 Not Found	404 Not Found	Pass	Proper error handling	View_problem_with_invalid_id
	3. POST /problems/:id/submit Submit solution with valid code (id:1)	Returns 201 Created with submission details	201 Created	Pass		Submit_solution

3.1. Submit solution with empty code (id:1)	Returns 400 bad Request	400 bad Request	Pass	Validation works correctly	Submit solution with empty code	
3.2. Submit solution as a non-contestant of the contest	Returns 403 error with message, “You are not participant in this contest”	201 Created	Failed	non - contestant can submit solutions	Submit as a non-contestant	
3.3. Submit solution with missing or invalid token	Returns message, “Unauthorized” 401 Unauthorized	401 Unauthorized	Pass	No unauthorized solution submits	Unauthorized submission	
3.4. Submit solution in an upcoming contest Contest starting timestamp: 2025-09-03T15:45:00Z Solution Submitting timestamp: 2025-09-03T15:41:57.987Z	400 bad Request {"message": "Contest is upcoming, submissions are not allowed yet"}	201 Created	Failed	Contestants are able to submit solutions to the upcoming contests	1. Contest Starting time 2. Submission time	
4. GET /problems/:id/submit Send To view the Problem submissions	Returns all submissions for problem (200 OK)	200 OK	Pass		View problem submissions	
4.1 Send with invalid problem ID to check problem’s submission which does not exist	Returns: {"message": “Problem not found”} (404 Not Found)	404 Not Found	Pass		View Problem Submissions with invalid problem id	

Title:	Result & Analysis					
Test Case ID	TC-06					
Tester Name	<i>Irfanuzzaman Montasir</i>					
Test case Description	To verify the functionality of view leaderboard after contest ends , check participation (valid user),fetch user's contest history,fetch user's submissions history.					
Pre-Condition	1.A contest with the given :id must exist. 2.The contest must be finished (end time has passed). 3.At least one participant must have submitted solutions. 4.A user with the given :userId must exist. 5.The user must have joined the contest. 6.The user must have participated in at least one contest. 7.The user must have submitted at least one solution to a problem.					
Test Scenario	In the Results & Analysis phase, we tested features related to contest outcomes and user performance. The scenarios included verifying that leaderboards are accessible after contests end, checking valid user participation details, and retrieving a user's contest history along with their submissions. These tests ensure accurate result tracking, proper data visibility, and reliable access to contest analytics.					
Test Case ID	Test Steps	Expected Result	Actual Result	Status	Comments	Evidence
TC-06	1. Send GET /contests/:id/leaderboard With valid Id	The leaderboard will be shown.	200 OK	Passed		https://drive.google.com/file/d/1wUxOfIXW-o-YE-jv31t0W2FgWXmEcdb/view?usp=drive_link
	1.1.Send GET /contests/:id/leaderboard With invalid id	The leaderboard won't be shown.	404 Not Found	Passed		https://drive.google.com/file/d/1tdWswhZtLvX-UOhLB5vmmGiDKtcdgvpw/view?usp=drive_link

1.2. Send GET/contests/:id/participation/ :userId with valid id & userId	Unique participated member Will be shown besides ranking.	200 OK	Passed		https://drive.google.com/file/d/1FUDjJEIM0r1FB4tAFXMiDBvc0iqvA07G/view?usp=drive_link
1.3 Send GET/contests/:id/participation/ :userId with invalid contest id and userId. This test can be done 2 ways, 1st. Valid contest id and invalid user name -. user not found will be shown 2nd. Invalid contest id and valid user id- contest not found will be shown.	1. user not found will be shown 2. contest not found will be shown.	404 Not Found	Passed		1. https://drive.google.com/file/d/1ajA3JZ_5D5iLRo9jLUaPUdiUUdW3bJ-W/view?usp=drive_link 2. https://drive.google.com/file/d/1TK-JPhbUfVwkzyLbX9G4RXqFqHFUqxca/view?usp=drive_link
1.4 Send GET/contests/:id/participation/ :userId	Contest Leaderboard won't be shown.	400 Bad request	Passed		https://drive.google.com/file/d/1pe_hsEHxL3b20UmuCys8Ilt7znXMek-/view?usp=drive_link
1.5 Send GET /users/:id/contests with valid user id	It will show the history of contests where the valid user has participated.	200 OK	Passed		https://drive.google.com/file/d/1LSykUmsDwO6yiQctUabF8kfmihYCYCQz/view?usp=drive_link

	3.2 Send GET /users/:id/contests with invalid user id	Users not found will be shown.	404 bad Request	Passed		https://drive.google.com/file/d/191CPZnqHHo7NeHEAuHBT5s_YOEolqdHP/view?usp=drive_link
	4. Send GET /users/:id/submissions With valid user id.	There is a fault in logic I guess, the empty [] array is showing.	200 Ok	Passed		https://drive.google.com/file/d/18MD82EKhpGdvjQk09ec-iZ3AjSyYBJbl/view?usp=drive_link
	4.1 Send GET /users/:id/submissions With invalid user id.	The submission array won't be shown.	404 Not Found	passed		https://drive.google.com/file/d/1dG3QAX_8lxMvHcukuGYYSMihGFHAcnIX/view?usp=drive_link

Title:	<i>Submission management</i>					
Test Case ID	TC-07					
Tester Name	Mahi hoque					
Test case Description	Testing user based submission and all submissions					
Pre-Condition	Valid contest with valid users making solution submission					
Test Scenario	Users can fetch all submissions or view details of a specific submission by a valid ID, both returning successful responses.					
Test Case ID	Test Steps	Expected Result	Actual Result	Status	Comments	Evidence

TC-07	1.1. GET /submissions all submissions that have been made	[{"id":1,"problemId":6,"userId":"b","verdict":"Accepted","timestamp":"2025-09-02T21:03:02.355Z"}, {"id":2,"problemId":7,"userId":"d","verdict":"Accepted","timestamp":"2025-09-02T21:03:43.274Z"}]	200	pass		https://drive.google.com/file/d/1t7lqLW3rDxOfLKWiwYICTTaL9QqMAiFw/view?usp=drive_link
--------------	---	--	-----	------	--	---

Title:	Problem Management					
Test Case ID	TC-08					
Tester Name	Md. Khademul Islam Nahin					
Test case Description	To verify the functionality of updating a problem.					
Pre-Condition	1.A problem with the given :id must exist. 2.The user making the request must be an admin. 3.Valid updated fields/data must be provided.					
Test Scenario	In the Problem Management phase, we tested the ability to update problem details. Admins were able to successfully modify problem information with valid data, while non-admin users were restricted.					
Test Case ID	Test Steps	Expected Result	Actual Result	Status	Comments	Evidence
TC-08	1.Send PUT /problems/:id With valid problem id & valid name,timeline.	The problem will be updated with its	200 OK.	passed		https://drive.google.com/file/d/1CLvbcjHDv0kqRoCjgP-ORHoD

		new name and time limit.				TvXL7_tN/view?usp=drive_link
	1.2 Send PUT /problems/:id With invalid problem id.	Problems not found will be shown.	404 Not Found.	Passed		https://drive.google.com/file/d/1ww35KonTf-le23VWTmjDrB7kmyUGZ2Q4/view?usp=drive_link

3. Defects

Defect ID: D-500-001

Title: **500** Internal Server Error on User Profile Update API - Missing JSON Body

Severity: Blocker

Steps to Reproduce:

1. Log in to the application with any valid user account.
2. Navigate to the user profile page.
3. Attempt to update any user profile field (e.g., name, email) and click "Save."

Expected: The API should return a **200 OK** status code with a JSON response confirming a successful update, or a clear **5xx** error with a descriptive JSON body if the update fails on the server.

Actual: The API returns a **500 Internal Server Error** with an empty response body. The client receives no information about the cause of the failure.

Logs/Responses:

- **Request:** POST to `/api/profile/update` with JSON payload (e.g., `{"name": "New Name"}`).
- **Response:**
 - Status Code: 500 Internal Server Error
 - Body: `{}` (empty)

Evidence:

- Screenshot of the network request showing the 500 status code and empty response ([Update Problem Error.PNG](#))
- Server-side log entry showing the unhandled exception (see [server_log.txt](#) attached in the previous report).

Defect ID: D-404-001

Title: 404 User Not Found on Contest Join API after Profile Update

Severity: Major

Steps to Reproduce:

1. Log in to the application with a valid user account.
2. Successfully register and authenticate with JWT.
3. Create or select a contest (valid contest ID).
4. Edit another valid account and change its user name
5. Attempt to join the contest using the endpoint POST `/contests/:id/join` with the valid bearer token.

Expected Result:

The API should allow the authenticated user to join the contest and Return:

```
{ "message": "User 3 joined contest abc", "contestId": 1 }
```

Actual Result:

The API returns a 404 Not Found error with:

```
{ "message": "User not found" }
```

even though the user is registered, logged in, and authenticated.

Logs/Responses:

- Request:

POST /contests/2/join

Authorization: Bearer <JWT>

- Response:

Status Code: 404 Not Found

Body: { "message": "User not found" }

Evidence:

- Screenshot of Postman request showing POST /contests/:id/join, valid bearer token, and the 404 User not found response ([contest join can't find user after profile edit.png](#)).
- Server-side log entry showing the unhandled exception (see [log.txt](#) attached in the previous report).

Defect ID: D-400-001

Title: 400 Bad Request on Contest Join API – Contest Marked as Ended Even When Running

Severity: Minor

Steps to Reproduce:

1. Log in with a valid user account and obtain a JWT.
2. Create or use a contest that is still ongoing (current time is within the contest start and end time).
3. Attempt to join the contest using the endpoint:

POST /contests/:id/join

Authorization: Bearer <JWT>

Expected Result:

The API should allow the authenticated user to join the contest and return:

```
{ "message": "User <username> joined contest <id>", "contestId": <id> }
```

Or,

The API should respond

Actual Result:

The API incorrectly rejects the request with:

```
{ "message": "Contest has already ended" }
```

despite the contest still being active.

Logs/Responses:

- Request:

POST /contests/2/join

Authorization: Bearer <JWT>

- Response:

Status Code: 400 Bad Request

Body: { "message": "Contest has already ended" }

Evidence:

- Screenshot of Postman request showing a 400 Bad Request response with "Contest has already ended" error for an active contest ([contest_join_join_after_start.png](#)).
- Server-side log entry showing the unhandled exception (see [log_response.txt](#) attached in the previous report).

Defect ID: D-200-001

Title: GET User Submissions Returns Empty List Despite Existing Submissions

Severity: Major

Steps to Reproduce:

1. Log in with a valid user and obtain a JWT.
2. Submit a solution to a contest problem using:

`POST /problems/:id/submit`

`Authorization: Bearer <JWT>`

`{ "problemId": <id>, "code": "print('hello')" }`

Ensure submission is accepted and stored.

3. Fetch the user's submissions with:

`GET /users/:id/submissions`

Expected Result:

The API should return a list of the user's submissions, including problemId, contestId, verdict, and timestamp.

Actual Result:

The API responds with `200 OK` but an empty submissions list:

```
{
  "submissions": []
}
```

even though the user has already submitted solutions.

Logs/Responses:

- **Request:**

`GET /users/2/submissions`

- **Response:**

`Status Code: 200 OK`

`Body: { "submissions": [] }`

Evidence:

- Screenshot of Postman showing `GET /users/:id/submissions` returning empty list despite successful submissions ([conteset_submission_by_user.png](#)).
- Server-side log entry showing the unhandled exception (see [log_script.txt](#) attached in the previous report).

Defect ID: D-403-001

Title: Non-participant user can submit solutions

Severity: Major

Steps to Reproduce:

1. Register/Login as a user who is **not joined** in the contest.
2. Call `POST /problems/:id/submit` with valid problemId and code.
3. Observe the response.

Expected Result:

403 Forbidden with {"message": "You are not participant in this contest"}

Actual Result:

201 Created with submission details.

Logs/Responses:

- Request: { "problemId": 1, "code": "print('hello')"

Authorization: Bearer <JWT>

- Response:

```
{
  "message": "Submission successful",
  "submission": { ... }
}
```

Evidence: (TC5 3.2 — Failed)

- User contest history Screenshot of Post man using `GET /users/contests`. ([User contest history.png](#))
- Screenshot of json body response of using trying to submit a solution using `POST /problems/submit` . ([Non-contestant can submit solution.png](#))

This is a **critical security/logic flaw**; breaks contest fairness.

Defect ID: D-400-002

Title: Submissions accepted before contest start time

Severity: Major

Steps to Reproduce:

1. Create a contest with a future start date/time
2. Add problems to this contest.
3. As a valid participant, submit a solution before the contest starts.

Expected Result:

400 Bad Request with {"message": "Contest is upcoming, submissions are not allowed yet"}

Actual Result:

201 Created with submission details.

Logs/Responses:

Request:

```
POST /problems/2/submit      Authorization: Bearer <valid_token>
{
  "problemId": 2,
  "code": "print('hello world')"
}
```

Response:

```
{
  "message": "Submission successful",
  "submission": { ... }
}
```

Contest start time: 2025-09-03T15:45:00Z

Evidence: (TC5 3.4 — Failed)

- Screenshot of postman response after creating the contest containing the contest start time using `POST /contests` ([contest starting time.png](#))
- Screenshot of postman response after submitting a solution containing the timestamp of submission which is before the start of the contest. ([submission time.png](#))

Notes: Contestants can submit solutions before the contest officially begins.

4. Risk and Recommendations

1. Weak Registration Process ([Evidence](#))

Description:

The system allows registration with only **username** and **password**, while other fields (**email**, **dateOfBirth**, **country**, **organization**) are optional.

Security Risks:

1. **Fake/Spam Accounts** – Attackers can create unlimited accounts without verification.
2. **No Identity Verification** – Difficult to confirm real users; impersonation is possible..
3. **High DoS Risk** – Automated bots can flood the system with fake registrations, exhausting resources and preventing real users from accessing the service.

Recommendations:

- Enforce mandatory fields (especially email).
- Implement **email/phone verification** during registration.
- Validate uniqueness of email/username to avoid impersonation.

2. Insecure Data Exposure

GET /users endpoint returns sensitive info (email, dateOfBirth, organization) without authentication → violates privacy and facilitates profiling attacks. (Evidence: **Data Exposure.PNG**)

3. Duplicate Email Registration Allowed

Risk

Allowing multiple users to register with the same email weakens **identity verification**, enables **account impersonation**, and makes **password recovery insecure**.

Recommendation:

Enforce **unique email constraints** in registration.

4. SQL injection

The registration system accepts raw user input without validation or sanitization. Malicious payloads such as SQL Injection (`admin'; DROP TABLE users; --`) and XSS (`<script>alert('XSS')</script>`) can be used in the `username` field.

Security Risks:

1. **SQL Injection** – Attackers can manipulate queries to **delete tables, read sensitive data, or escalate privileges**.
2. **Cross-Site Scripting (XSS)** – Malicious scripts may execute in other users' browsers, leading to **session hijacking, credential theft, or defacement**.
3. **Data Integrity Loss** – Risk of corrupted or destroyed database records.
4. **System Compromise** – Exploitation can extend to full server compromise if not contained.

Recommendations:

- Implement **input validation** (allow only expected characters for username).
- Use **prepared statements / parameterized queries** for database operations.
- Apply **output encoding** (escape HTML before rendering).
- Enable **Web Application Firewall (WAF)** rules against injection patterns.

All risk related evidence has been provided in the Risk folder.